



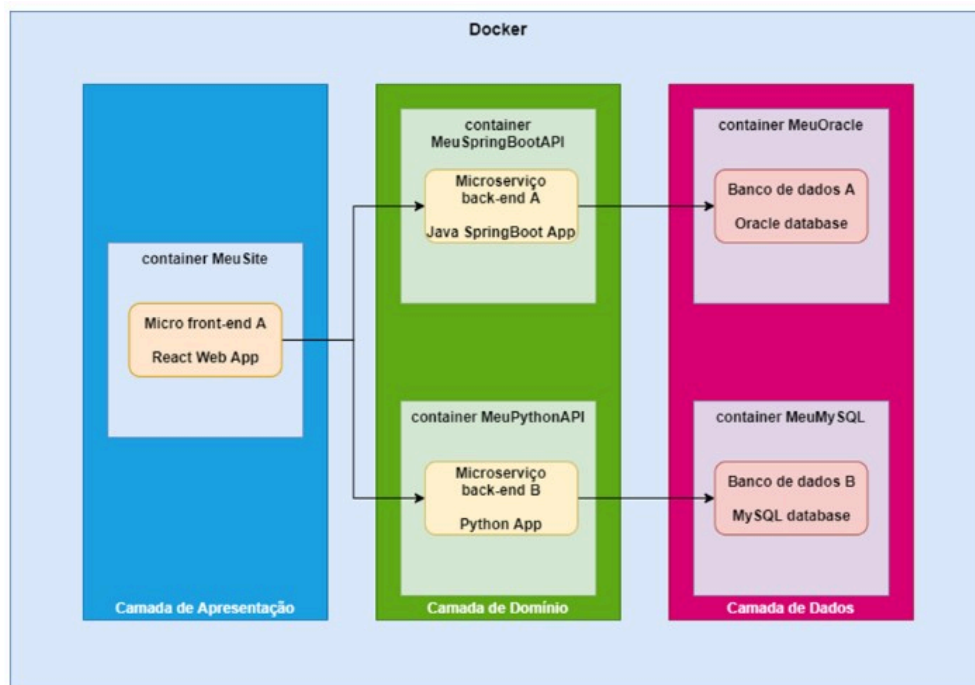
Subindo uma solução completa

Finalmente chegamos para falar do conjunto de soluções que usaremos como modelo para aprofundar no estudo do Docker.

A primeira coisa a se fazer é baixar o repositório que preparei, neste link: <https://gitlab.com/evertton.juniti/descomplica>

No próprio repositório tem várias instruções de como clonar o repositório e como criar algumas coisas, mas vamos com calma.

Seguiremos o seguinte desenho:

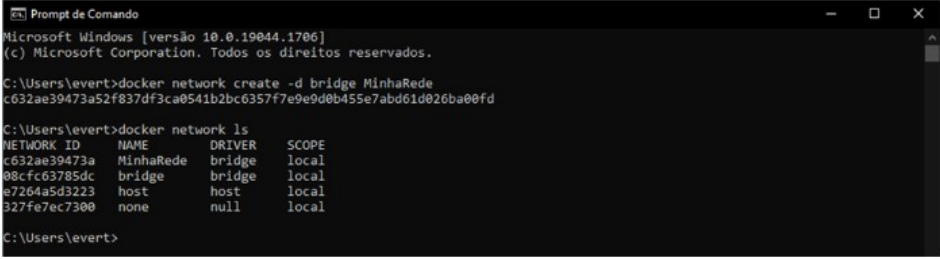


Passaremos por alguns conceitos para conseguir subir essa solução toda, mas a primeira coisa a fazermos é criar uma “sub-rede” no Docker. Já vimos o potencial de

“sub-rede” e a comunicação com o uso de portas e é justamente isso que faremos com essa solução.

Vamos primeiramente criar uma “sub-rede”, para isso podemos usar o seguinte comando:

`docker network create -d bridge MinhaRede`



```
Prompt de Comando
Microsoft Windows [versão 10.0.19044.1706]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\event>docker network create -d bridge MinhaRede
c632ae39473a52f837df3ca0541b2bc6357f7e9e9d0b455e7abd61d026ba00fd

C:\Users\event>docker network ls
NETWORK ID        NAME        DRIVER        SCOPE
c632ae39473a      MinhaRede   bridge        local
08cfc63785dc      bridge     bridge        local
e7264a5d3223      host       host          local
327fe7ec7300      none       null          local
C:\Users\event>
```

“docker network” é o comando do Docker para lidarmos com tudo de redes, o comando “create” indica que estamos criando uma rede (uma “sub-rede” no caso), o parâmetro “-d bridge” indica o DRIVER que usaremos que é o tipo bridge que permite a comunicação livre entre containers e o último parâmetro “MinhaRede” é um apelido para a nossa rede.

Recapitulando esse conceito, estamos criando uma “sub-rede” para que as APIs que iremos criar consigam se comunicar com os bancos de dados. Se não fizermos isso a API não “enxergará” esses bancos de dados.

Observação: em ambiente produtivo há outros mecanismos para fazer os containers enxergarem uns aos outros, mas para a prática que estamos montando aqui que será tudo em um único computador (o seu) criamos este artifício para facilitar toda a configuração.

“sub-rede” criada, então agora podemos subir as soluções. Como front-end vai falar com API e a API por sua vez vai falar com o banco de dados, é necessário que essas dependências subam primeiro para que as aplicações não dêem problema na hora

de serem executadas. Por este motivo subiremos primeiramente os bancos de dados.

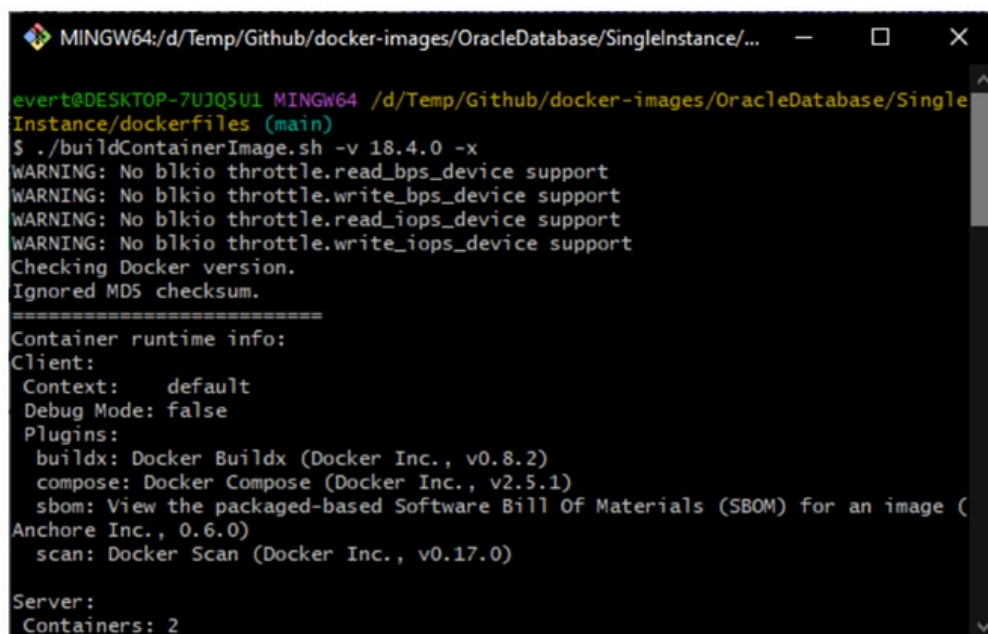
Vamos subir primeiro o banco de dados Oracle! As instruções completas estão no link: https://gitlab.com/evertton.juniti/descomplica/-/tree/main/cicd_database/oracle

Neste momento irei resumir os passos, então primeiramente iremos fazer o build de uma imagem do Oracle. O Oracle possui imagens oficiais, porém não são públicas e por este motivo há a necessidade de clonarmos o repositório oficial do Oracle e escolher uma das versões para fazermos o build. Então neste momento, devemos fazer o clone do seguinte repositório: <https://github.com/oracle/docker-images.git>

No local onde você fez o clone do repositório em sua máquina, entre no conjunto de pastas “docker-images\OracleDatabase\SingleInstance\dockerfiles” e abra um terminal bash do Git (o Git Bash mesmo), depois execute o seguinte comando:

```
./buildContainerImage.sh -v 18.4.0 -x
```

Dica: digite o comando mesmo, se você copiar o comando e colar no bash, vai dar um erro e não vai fazer o build! Vai levar bastante tempo para a imagem ser criada, neste momento você terá que ser um pouco paciente.



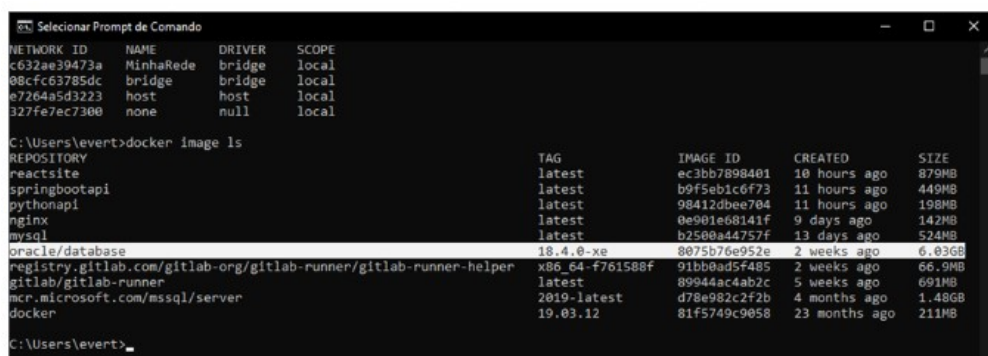
```

MINGW64:/d/Temp/Github/docker-images/OracleDatabase/SingleInstance/...
evert@DESKTOP-7UJQ5U1 MINGW64 /d/Temp/Github/docker-images/OracleDatabase/SingleInstance/dockerfiles (main)
$ ./buildContainerImage.sh -v 18.4.0 -x
WARNING: No blkio throttle.read_bps_device support
WARNING: No blkio throttle.write_bps_device support
WARNING: No blkio throttle.read_iops_device support
WARNING: No blkio throttle.write_iops_device support
Checking Docker version.
Ignored MD5 checksum.
=====
Container runtime info:
Client:
Context:    default
Debug Mode: false
Plugins:
buildx: Docker Buildx (Docker Inc., v0.8.2)
compose: Docker Compose (Docker Inc., v2.5.1)
sbom: View the packaged-based Software Bill Of Materials (SBOM) for an image (Anchore Inc., 0.6.0)
scan: Docker Scan (Docker Inc., v0.17.0)

Server:
Containers: 2

```

Depois que terminar o build, você verá que uma imagem nova apareceu em seu repositório local chamado “oracle/database”:



```

Selecionar Prompt de Comando
NETWORK ID NAME DRIVER SCOPE
c632ae39473a MinhaRede bridge local
08cfc63785dc bridge local
e7264a5d3223 host host local
327fe7ec7300 none null local

C:\Users\evrt>docker image ls
REPOSITORY TAG IMAGE ID CREATED SIZE
reactsite latest ec3bb7898401 10 hours ago 879MB
springbootapi latest b9f5eb1c6f73 11 hours ago 449MB
pythonapi latest 98412dbee704 11 hours ago 198MB
nginx latest 0e901e68141f 9 days ago 142MB
mysql latest b2500a44757f 13 days ago 524MB
oracle/database 18.4.0-xe 8075b76e952e 2 weeks ago 6.03GB
registry.gitlab.com/gitlab-org/gitlab-runner/gitlab-runner-helper x86_64-f761588f 91bb0ad5f485 2 weeks ago 66.9MB
gitlab/gitlab-runner latest 89944ac4ab2c 5 weeks ago 691MB
mcr.microsoft.com/mssql/server 2019-latest d78e982c2f2b 4 months ago 1.48GB
docker 19.03.12 81f5749c9058 23 months ago 211MB

C:\Users\evrt>

```

Antes de subir o container, prepare um conjunto de pastas para usarmos volume no container. Assim guardaremos todos os dados do banco de dados em um local específico. Crie o conjunto de pastas “oracle\oradata” e “oracle\setup” no local de sua preferência. No caso da pasta “oracle\setup”, inclua nela o arquivo “script_inicial.sql” do repositório.

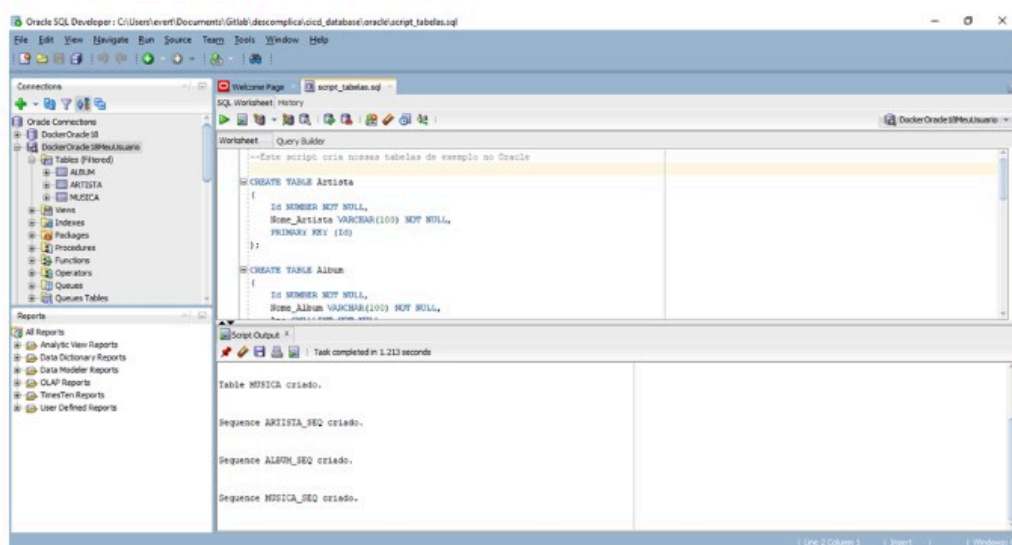
Em seguida agora sim você poderá executar o seguinte comando para subir um banco de dados Oracle:

```
docker run --name MeuOracle --network MinhaRede -v
/D/docker/volumes/oracle/oradata:/opt/oracle/oradata -v
/D/docker/volumes/oracle/setup:/opt/oracle/scripts/setup -p 1521:1521 -p 5500:5500
-e ORACLE_PWD=MinhaSenha -e ORACLE_CHARACTERSET=AL32UTF8 -d
oracle/database:18.4.0-xe
```

Os detalhes de cada parâmetro estão nas instruções do link: https://gitlab.com/evertton.juniti/descomplica/-/tree/main/cicd_database/oracle

Na primeira vez que subir o Oracle, levará bastante tempo para ele fazer a configuração inicial (aqui levou de 30 a 40 minutos).

Agora vamos nos conectar com o Oracle SQL Developer, as instruções de como instalar a ferramenta e configurar a conexão com o usuário criado no “script_inicial.sql” chamado MeuUsuario estão no link: https://gitlab.com/evertton.juniti/descomplica/-/tree/main/cicd_database/oracle. Vamos executar o script “script_tabelas.sql” que está neste repositório, isso criará nossas tabelas de exemplo:



Agora vamos subir o banco de dados MySQL! As instruções completas estão no link: https://gitlab.com/evertton.juniti/descomplica/-/tree/main/cicd_database/mys

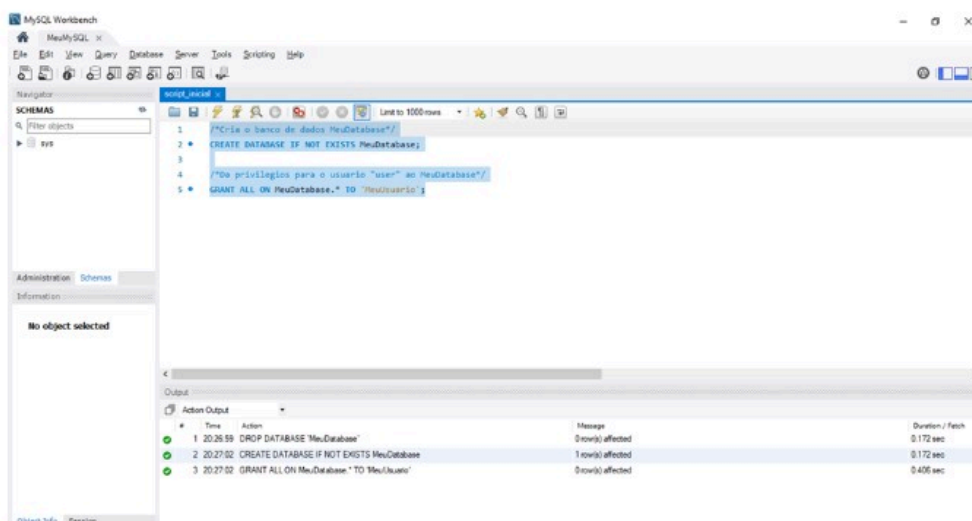
ql

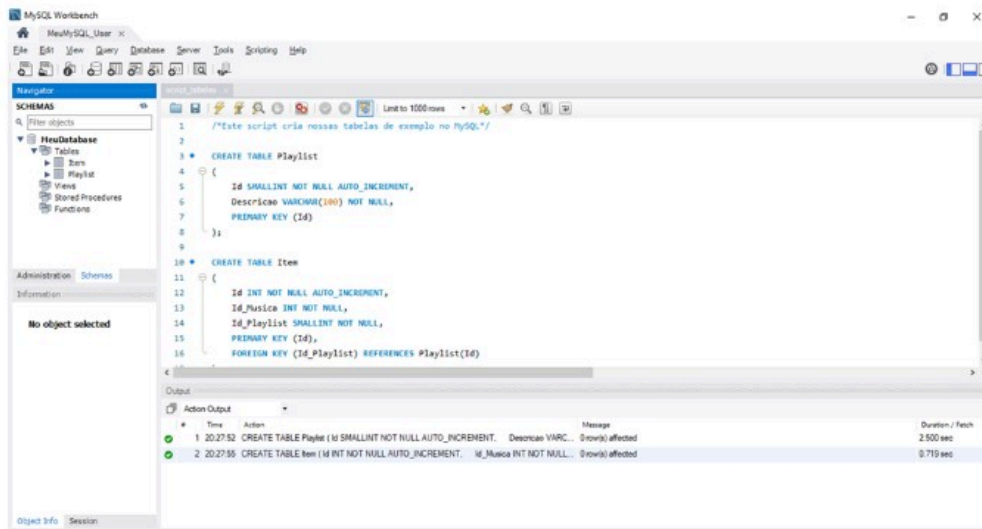
Aqui iremos resumir, então crie uma pasta chamada “mysql” em um local que você deseje, depois podemos executar o seguinte comando para subir um container:

```
docker run --name MeuMySQL --network MinhaRede -v /D/docker/volumes/mysql:/var/lib/mysql -p 3306:3306 -e MYSQL_ROOT_PASSWORD=Minha@senha -e MYSQL_USER=MeuUsuario -e MYSQL_PASSWORD=MinhaSenha -d mysql:latest
```

Os detalhes de cada parâmetro estão nas instruções do link: https://gitlab.com/everton.juniti/descomplica/-/tree/main/cicd_database/mysql

Agora vamos nos conectar com o MySQL Workbench as instruções de como instalar a ferramenta e configurar a conexão com o usuário root e com o usuário customizado criado chamado e MeuUsuario estão no link: https://gitlab.com/everton.juniti/descomplica/-/tree/main/cicd_database/mysql. Vamos executar o script “script_inicial.sql” primeiramente com o usuário root para criar o schema novo e dar permissão ao usuário MeuUsuario que criamos e logo em seguida executaremos o script “script_tabelas.sql” que está neste repositório também só que com o usuário MeuUsuario, isso criará nossas tabelas de exemplo:





Agora vamos subir nossos back-ends! Primeiro vou começar com a API em Java SpringBoot que se conecta ao banco de dados Oracle. As instruções completas estão no link: https://gitlab.com/evertton.juniti/descomplica/-/tree/main/cicd_backend/springbootapi

Neste caso já há código pronto! Iremos fazer o build do projeto para criar uma imagem desta aplicação customizada e depois subir um container. Vamos olhar o Dockerfile desta solução em Java:

```
#
# Build stage
#
FROM maven:3.8.5-openjdk-17-slim AS build
COPY src /home/app/src
RUN rm -rf /home/app/src/main/resources/application.properties
COPY ./applicationProd.properties
/home/app/src/main/resources/application.properties
COPY pom.xml /home/app
RUN mvn -f /home/app/pom.xml clean package

#
# Package stage
#
FROM openjdk:17-slim
COPY --from=build /home/app/target/springbootapi-1.0.0.jar
/usr/local/lib/springbootapi.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/usr/local/lib/springbootapi.jar"]
```


Resumidamente, o Dockerfile instrui o uso de uma imagem base do Java especificamente para efetuar a compilação do nosso código em java, neste caso a imagem é a “maven:3.8.5-openjdk-17-slim”. Como o projeto do Java utiliza o repositório do Maven para obter as bibliotecas necessárias para o projeto, usamos essa imagem base do Maven com Java para compilar o projeto. Após a compilação do projeto e geração do arquivo “.jar”, temos a parte final que usa uma imagem base “openjdk:17-slim” especificamente para executar nosso projeto compilado quando um container for criado à partir desta imagem. É exposta a porta 8080 como porta do container para receber as requisições, note que o comando para executar nossa aplicação quando o container é iniciado é o seguinte: “java -jar /usr/local/lib/springbootapi.jar”.

O detalhamento de cada linha deste Dockerfile se encontra no link: https://gitlab.com/evertton.juniti/descomplica/-/tree/main/cicd_backend/springbootapi

Podemos usar o seguinte comando para fazer o build da nossa aplicação API Java SpringBoot:

```
docker build --label springbootapi --no-cache -t springbootapi:latest .
```

Os detalhes de cada parâmetro estão nas instruções do link: https://gitlab.com/evertton.juniti/descomplica/-/tree/main/cicd_backend/springbootapi


```

C:\Windows\System32\cmd.exe
C:\Users\evert\Documents\Gitlab\descomplica\cicd_backend\springbootapi>docker build --label springbootapi --no-cache -t
springbootapi:latest .
[+] Building 253.2s (16/16) FINISHED
=> [internal] load build definition from Dockerfile 2.6s
=> => transferring dockerfile: 32B 0.2s
=> [internal] load .dockerignore 3.6s
=> => transferring context: 2B 0.2s
=> [internal] load metadata for docker.io/library/openjdk:17-slim 5.9s
=> [internal] load metadata for docker.io/library/maven:3.8.5-openjdk-17-slim 5.2s
=> [auth] library/maven:pull token for registry-1.docker.io 0.0s
=> [auth] library/openjdk:pull token for registry-1.docker.io 0.0s
=> CACHED [build 1/6] FROM docker.io/library/maven:3.8.5-openjdk-17-slim@sha256:582e781d39f0b40fbd02eb23f5b76636 0.0s
=> CACHED [stage-1 1/2] FROM docker.io/library/openjdk:17-slim@sha256:aaa3b3cb27e3e528b8f116863d0588c438ed55ecfa 0.0s
=> [internal] load build context 2.7s
=> => transferring context: 77.46kB 0.8s
=> [build 2/6] COPY src /home/app/src 3.7s
=> [build 3/6] RUN rm -rf /home/app/src/main/resources/application.properties 0.2s
=> [build 4/6] COPY ./applicationProd.properties /home/app/src/main/resources/application.properties 4.7s
=> [build 5/6] COPY pom.xml /home/app 4.8s
=> [build 6/6] RUN mvn -f /home/app/pom.xml clean package 100.8s
=> [stage-1 2/2] COPY --from=build /home/app/target/springbootapi-1.0.0.jar /usr/local/lib/springbootapi.jar 7.0s
=> exporting to image 5.8s
=> => exporting layers 4.2s
=> => writing image sha256:f35e7e525217f8b094db201c6511f375fd11e193d2393c18b10ee7c043d2824a 0.3s
=> => naming to docker.io/library/springbootapi:latest 0.5s

Use 'docker scan' to run Snyk tests against images to find vulnerabilities and learn how to fix them
C:\Users\evert\Documents\Gitlab\descomplica\cicd_backend\springbootapi>

```

E por fim podemos criar um container com o seguinte comando:

```
docker run --name MeuSpringBootAPI --network MinhaRede -p 8080:8080 -d
springbootapi:latest
```

Os detalhes de cada parâmetro estão nas instruções do link: https://gitlab.com/everton.juniti/descomplica/-/tree/main/cicd_backend/springbootapi

Agora vamos subir nossa segunda aplicação back-end, nosso API Python FastAPI que conecta ao banco de dados MySQL. As instruções completas estão no link: https://gitlab.com/everton.juniti/descomplica/-/tree/main/cicd_backend/pythonapi

Neste caso já há código pronto! Iremos fazer o build do projeto para criar uma imagem desta aplicação customizada e depois subir um container. Vamos olhar o Dockerfile desta solução em Python:

```
FROM python:3.6-slim

WORKDIR /code
COPY ./requirements.txt /code/requirements.txt

RUN pip install --no-cache-dir --upgrade -r /code/requirements.txt

COPY ./app /code/app
RUN rm -rf /code/app/data/databaseurl.py
COPY ./databaseurlprod.py /code/app/data/databaseurl.py

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

No caso do Python, não há necessidade de se compilar o código, então este Dockerfile apenas indica o uso da imagem base “python:3.6-slim” e demais códigos que trazem para dentro da imagem todas as bibliotecas usadas no projeto e seus códigos-fonte. É exposta a porta 8000 como porta do container para receber as requisições, note que o comando para executar nossa aplicação quando o container é iniciado é o seguinte: “uvicorn app.main:app --host 0.0.0.0 --port 8000”.

O detalhamento de cada linha deste Dockerfile se encontra no link: https://gitlab.com/everton.juniti/descomplica/-/tree/main/cicd_backend/pythonapi

Podemos usar o seguinte comando para fazer o build da nossa aplicação API Python FastAPI:

```
docker build --label pythonapi --no-cache -t pythonapi:latest .
```

Os detalhes de cada parâmetro estão nas instruções do link: https://gitlab.com/everton.juniti/descomplica/-/tree/main/cicd_backend/pythonapi

```

C:\Windows\System32\cmd.exe
Microsoft Windows [versão 10.0.19044.1706]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\evert\Documents\Gitlab\descomplica\cicd_backend\pythonapi>docker build --label pythonapi --no-cache -t pythonapi:latest .
[+] Building 112.2s (13/13) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 393B
=> [internal] load .dockerignore
=> => transferring context: 128B
=> [internal] load metadata for docker.io/library/python:3.6-slim
=> [auth] library/python:pull token for registry-1.docker.io
=> [internal] load build context
=> => transferring context: 12.34kB
=> [1/7] FROM docker.io/library/python:3.6-slim@sha256:2cf5bc27956e6a55f78606864d91fe527696f9e32a724e6f9702b5f96
=> CACHED [2/7] WORKDIR /code
=> [3/7] COPY ./requirements.txt /code/requirements.txt
=> [4/7] RUN pip install --no-cache-dir --upgrade -r /code/requirements.txt
=> [5/7] COPY ./app /code/app
=> [6/7] RUN rm -rf /code/app/data/databaseurl.py
=> [7/7] COPY ./databaseurlprod.py /code/app/data/databaseurl.py
=> exporting to image
=> => exporting layers
=> => writing image sha256:9ecb2ce8b997ee60e3121caeb3d5ee21be12f7edbbe52202acfb92d5a390c8ac
=> => naming to docker.io/library/pythonapi:latest

Use 'docker scan' to run Snyk tests against images to find vulnerabilities and learn how to fix them

C:\Users\evert\Documents\Gitlab\descomplica\cicd_backend\pythonapi>_

```

E por fim podemos criar um container com o seguinte comando:

```
docker run --name MeuPythonAPI --network MinhaRede -p 8000:8000 -d pythonapi:latest
```

Os detalhes de cada parâmetro estão nas instruções do link: https://gitlab.com/everton.juniti/descomplica/-/tree/main/cicd_backend/pythonapi

E por último vamos dar uma olhada em nosso site feito com React, que conecta nas duas APIs que subimos, em Java e em Python! As instruções completas estão no link: https://gitlab.com/everton.juniti/descomplica/-/tree/main/cicd_frontend/meusite

Vamos dar uma olhada no Dockerfile:

```

FROM node:slim
WORKDIR /app
ENV PATH /app/node_modules/.bin:$PATH
COPY ./src /app/src
COPY ./public /app/public
COPY package.json /app/package.json
RUN npm install
RUN npm install react-scripts@3.3.1 -g
EXPOSE 3000
CMD ["npm", "start"]

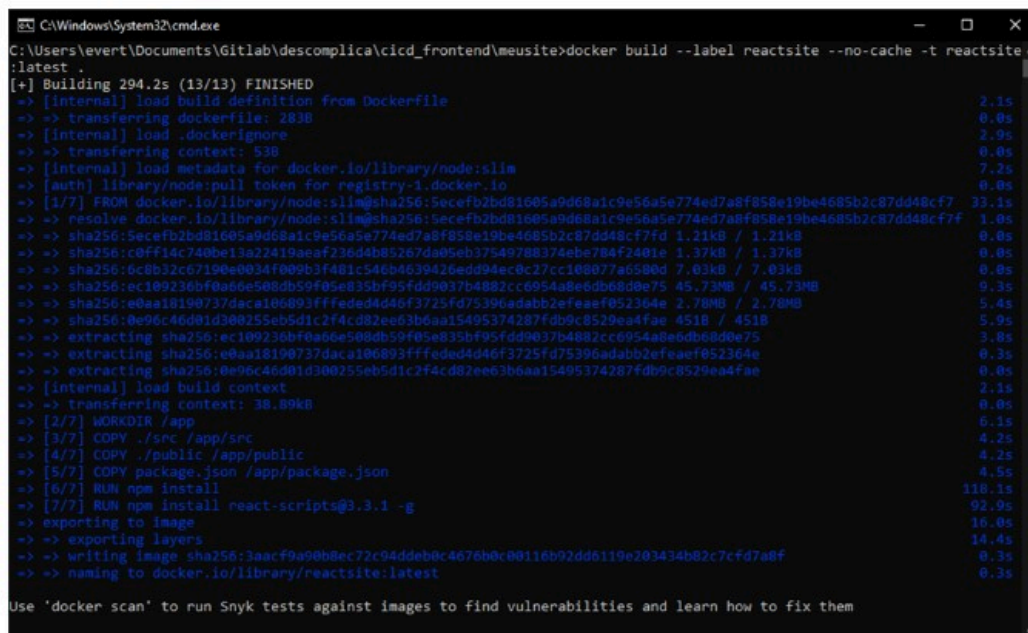
```

O detalhamento de cada linha deste Dockerfile se encontra no link: https://gitlab.com/everton.juniti/descomplica/-/tree/main/cicd_frontend/meusite

Podemos usar o seguinte comando para fazer o build da nossa aplicação React:

```
docker build --label reactsite --no-cache -t reactsite:latest .
```

Os detalhes de cada parâmetro estão nas instruções do link: https://gitlab.com/everton.juniti/descomplica/-/tree/main/cicd_frontend/meusite



```
C:\Windows\System32\cmd.exe
C:\Users\evert\Documents\Gitlab\descomplica\cicd_frontend\meusite>docker build --label reactsite --no-cache -t reactsite:latest .
[+] Building 294.2s (13/13) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 283B 2.1s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 53B 0.0s
=> [internal] load metadata for docker.io/library/node:slim 7.2s
=> [auth] library/node:pull token for registry-1.docker.io 0.0s
=> [1/7] FROM docker.io/library/node:slim@sha256:5ecef2b2bd81605a9d68a1c9e56a5e774ed7a8f058e19be4685b2c87dd48cf7f 33.1s
=> resolve docker.io/library/node:slim@sha256:5ecef2b2bd81605a9d68a1c9e56a5e774ed7a8f058e19be4685b2c87dd48cf7f 1.0s
=> sha256:5ecef2b2bd81605a9d68a1c9e56a5e774ed7a8f058e19be4685b2c87dd48cf7f 1.21kB / 1.21kB 0.0s
=> sha256:c0ff14c740be13a22419aef236d4b85267da05eb37549788374ebe784f2401e 1.37kB / 1.37kB 0.0s
=> sha256:6c8b32c67190e0034f009b3f481c546b4639426edd94ec0c27cc180077a6580d 7.03kB / 7.03kB 0.0s
=> sha256:ec109236bf0a6de508db59f05e835bf95fdd9037b4882cc6954a8e6db68d0e75 45.73MB / 45.73MB 9.3s
=> sha256:e0aa18190737daca106893fffeded4d46f3725fd75396adabb2efaeaf052364e 2.78MB / 2.78MB 5.4s
=> sha256:e096c46d01d300255eb5d1c2f4cd82ee63b6aa15495374287fdb9c8529ea4fae 451B / 451B 5.0s
=> extracting sha256:ec109236bf0a6de508db59f05e835bf95fdd9037b4882cc6954a8e6db68d0e75 3.0s
=> extracting sha256:e0aa18190737daca106893fffeded4d46f3725fd75396adabb2efaeaf052364e 0.3s
=> extracting sha256:e096c46d01d300255eb5d1c2f4cd82ee63b6aa15495374287fdb9c8529ea4fae 0.0s
=> [internal] load build context 2.1s
=> => transferring context: 38.89kB 0.0s
=> [2/7] WORKDIR /app 6.1s
=> [3/7] COPY ./src /app/src 4.2s
=> [4/7] COPY ./public /app/public 4.2s
=> [5/7] COPY package.json /app/package.json 4.5s
=> [6/7] RUN npm install 110.1s
=> [7/7] RUN npm install react-scripts@3.3.1 -g 92.0s
=> exporting to image 16.0s
=> => exporting layers 14.4s
=> writing image sha256:3aac9fa90b8ec72c94ddeb0c4676b0c0116b92dd6119e203434b82c7cfd7a8f 0.3s
=> naming to docker.io/library/reactsite:latest 0.3s

Use 'docker scan' to run Snyk tests against images to find vulnerabilities and learn how to fix them
```

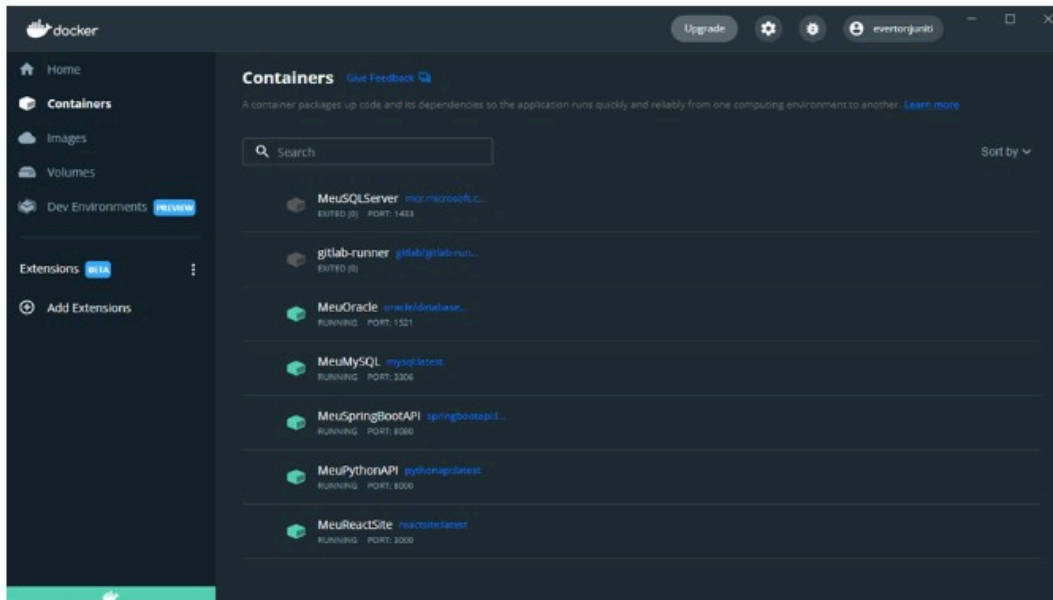
E por fim podemos criar um container com o seguinte comando:

```
docker run --name MeuReactSite --network MinhaRede -p 3000:3000 -d reactsite:latest
```

Os detalhes de cada parâmetro estão nas instruções do link: https://gitlab.com/everton.juniti/descomplica/-/tree/main/cicd_frontend/meusite

Depois de tudo isso, podemos ver que todos os containers estão de pé: os bancos de dados Oracle (na porta 1521) e MySQL (na porta 3306), as APIs Java (na porta 8080) e Python (na porta 8000) e nosso site React (na porta 3000).

O site se comunica com as APIs e as APIs se comunicam com os bancos de dados!



Podemos usar o seguinte comando para verificar que todos esses containers estão na nossa “sub-rede” chamada “MinhaRede”:

```
docker network inspect MinhaRede
```

```
Prompt de Comando

C:\Users\event>docker network inspect MinhaRede
[
  {
    "Name": "MinhaRede",
    "Id": "c632ae39473a52f837df3ca0541b2bc6357f7e9e9d0b455e7abd61d026ba00fd",
    "Created": "2022-06-06T22:23:30.911906953Z",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": {},
      "Config": [
        {
          "Subnet": "172.18.0.0/16",
          "Gateway": "172.18.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Containers": {
      "08cd1ff7efefda40c366f5ea0baa8b512f62142245f60fca264fbd9df609a82b": {
        "Name": "MeuMySQL",
        "EndpointID": "abc9ecec19591396122611f4dc0b45446618e29c1360157fdd00f688ca988087",
        "MacAddress": "02:42:ac:12:00:03",
        "IPv4Address": "172.18.0.3/16",
        "IPv6Address": ""
      },
      "68dc56005f5abfb4326b9389e9994c37129bc3c01a3d0409ae315d344668dcf3": {
        "Name": "MeuReactSite",
        "EndpointID": "dc34810c05e56511db63c2eeb0e27a1db5cc32d7309a7d65cee81f0ea1d887e3",
        "MacAddress": "02:42:ac:12:00:06",
        "IPv4Address": "172.18.0.6/16",
        "IPv6Address": ""
      }
    }
  }
]
```

```
Prompt de Comando

    },
    "91b8139063b848e839026bba7dc437319bcf46ca5563154dfbf78cf41a0d8817": {
      "Name": "MeuPythonAPI",
      "EndpointID": "bf55f27c8182ef44f8993e73c11a5616d37221f0831fb0af55fd234b2c428a93",
      "MacAddress": "02:42:ac:12:00:05",
      "IPv4Address": "172.18.0.5/16",
      "IPv6Address": ""
    },
    "dca3d07b372e9bb5af2f5f175dca066d6c0efefb7b5cab0f7d5193cecd4271e1": {
      "Name": "MeuOracle",
      "EndpointID": "6c6094dd96abeab15bef9b31f949db54cc07586328faa2ed0e3989e51825c72",
      "MacAddress": "02:42:ac:12:00:02",
      "IPv4Address": "172.18.0.2/16",
      "IPv6Address": ""
    },
    "e5fa9d61df46e8ac71939546aa03d5e233414ecf37efc9150ec79c1e8b858d4d": {
      "Name": "MeuSpringBootAPI",
      "EndpointID": "dd98663243379fbf1f8cc4f882e0c763a1fd46346a6c4b66483ea295cef451e9",
      "MacAddress": "02:42:ac:12:00:04",
      "IPv4Address": "172.18.0.4/16",
      "IPv6Address": ""
    }
  },
  "Options": {},
  "Labels": {}
}

C:\Users\event>
```

Atividade Extra

Para se aprofundar no assunto desta aula leia no detalhe o conteúdo todo do repositório: “Descomplica” no GitLab, de Everton Juniti Ogura.

Link do repositório: <https://gitlab.com/everton.juniti/descomplica>

Referência Bibliográfica

- OGURA, Everton J. Repositório do GitLab, projeto Descomplica. Disponível em <https://gitlab.com/everton.juniti/descomplica>. Acesso em 06 de junho de 2022.

Ir para exercício